

FitBuddy AI

2. Requirement Analysis

Functional and non-functional requirements for FitBuddy AI.

Functional Requirements

- Accept user fitness inputs through a web form.
- Validate required inputs before generating a plan.
- Create a structured prompt from the submitted profile.
- Send the request to the configured Google Gemini service.
- Receive and process the generated response.
- Display workout and nutrition results in the browser.
- Handle invalid input and AI/API errors gracefully.

Suggested User Inputs

- Age or age range, when required by the implementation.
- Fitness goal such as fat loss, general fitness or muscle development.
- Experience level.
- Available workout environment/equipment.
- Preferred workout frequency.
- Relevant dietary preference or restrictions.
- Other project-specific fields implemented in the form.

Non-Functional Requirements

- Usability: forms and generated plans should be easy to understand.
- Performance: requests should complete within a reasonable response time.
- Reliability: failures from the AI service should not crash the application.
- Security: API keys and secrets must never be committed to GitHub.
- Maintainability: routes, templates, static assets and configuration should be separated clearly.

Technology Requirements

- Python.
- FastAPI.
- Jinja2 templates.
- Google Gemini API/model integration as configured by the project.
- HTML/CSS/JavaScript for the web interface.
- A Python dependency file such as requirements.txt.

Acceptance Criteria

- A user can submit valid profile information.
- The backend receives and validates the request.

- The AI request is generated using the submitted data.
- A workout and nutrition response is displayed.
- Invalid input produces a useful error message.
- No secret API key is exposed in source control.